

RAG 第三阶段：Prompt + Gemini + 最终回答

现在开始开发 RAG 项目的第三阶段。

第一阶段和第二阶段已经完成。

现在把：

Document Indexing

和

Retrieval

连接起来，完成完整 RAG 闭环。

这是本项目最后一个开发阶段。

完成本阶段后项目即结束。

不要开发第四阶段。

一、本阶段目标

实现：

用户问题 → Query Embedding → Vector Search → Top-K Context → RAG Prompt → Gemini LLM → AI Answer

最终用户可以在右侧 Chat 区域直接向自己的知识库提问。

二、Chat API

实现：

POST /api/chat

请求：

```
{ "query": "...", "topK": 5 }
```

根据实际项目结构，可以增加：

- selectedDocumentIds
- temperature

但不要增加不必要参数。

三、完整 RAG Pipeline

Chat API 内部：

第一步：

接收用户 Query。

第二步：

调用现有 Embedding Service。

第三步：

调用现有 Retriever。

第四步：

获得 Top-K Chunk。

第五步：

构建 RAG Context。

第六步：

构建 Prompt。

第七步：

调用 Gemini LLM。

第八步：

返回：

- answer
 - sources
 - retrievalResults
-

四、Prompt

创建：

rag/prompt.js

不要把 Prompt 字符串直接写进 Chat Route。

Prompt 必须明确告诉 Gemini：

1. 你是一个知识库问答助手。
2. 优先根据提供的 Context 回答。
3. 不要把 Context 中不存在的信息伪装成知识库内容。
4. 如果 Context 无法回答问题，要明确告诉用户。
5. 可以使用模型自身常识帮助理解，但不能假装这些内容来自知识库。
6. 尽量引用相关文档。
7. 回答清晰、准确。
8. 使用 Markdown。

Context 与用户问题必须明确区分。

五、Sources

最终 AI Answer 必须返回来源。

例如：

```
{"answer": "...", "sources": [{ "filename": "xxx.pdf", "chunkIndex": 3, "score": 0.87 }]}
```

前端可以在 AI 回答下面展示：

“参考资料”

并列出本次回答使用的文件和 Chunk。

六、前端 Chat UI

右侧正式启用完整 Chat。

上方：

显示用户问题和 AI 回答。

AI 回答支持：

- Markdown
- 代码块
- 列表
- 标题

底部：

固定输入框。

用户发送后：

1. 输入框进入 loading。
2. 禁止重复发送。
3. 显示 AI 思考 / 加载状态。
4. 请求 `/api/chat`。
5. 返回 answer。
6. 展示 sources。
7. 恢复输入状态。

错误时：

显示用户能够理解的错误信息。

七、是否显示 Retrieval Context

建议在 UI 中保留一个可以展开的：

“检索到的知识”

区域。

默认可以折叠。

用户展开后可以看到：

- 文件名
- Chunk
- similarity score
- 被送给 Gemini 的 Context

这样这个项目可以非常直观地证明：

AI 的回答不是简单调用 Gemini，而是经过 RAG Retrieval 得到知识后再生成。

这对学习和面试展示非常重要。

八、RAG 参数

左侧保留：

Chunk Size

Chunk Overlap

Top K

可以根据实际情况提供：

Temperature

但 Temperature 不是 RAG 核心参数，如果实现后让界面过于复杂，可以不提供。

参数必须有默认值。

不要让普通用户看到大量模型内部参数。

九、Gemini Service

继续使用：

services/gemini.js

统一管理：

- Embedding
- Chat / Generate Content

如果你认为 Embedding 和 LLM 调用应该拆成：

embedding.js

gemini.js

也可以。

但是职责必须清晰。

不要在多个文件复制 Gemini API 请求代码。

十、错误处理

至少处理：

- Gemini API 失败
- Vectorize 查询失败
- Query Embedding 失败
- 没有检索结果
- 文件不存在
- 参数错误
- 网络错误
- 超时

如果没有相关 Context：

不要强行生成一个看起来非常确定的答案。

应该告诉用户：

“知识库中没有找到足够相关的资料。”

然后根据 Prompt 决定是否允许模型提供有限的常识解释。

十一、最终测试

必须进行完整端到端测试：

测试文档：

上传一个包含明确事实的文档。

例如文档中写：

“Jason 的 RAG 项目使用 Gemini 生成 Embedding，使用 Cloudflare Vectorize 保存向量。”

然后询问：

“这个 RAG 项目使用什么向量数据库？”

预期：

系统首先检索到相关 Chunk。

然后 Gemini 根据 Context 回答：

使用 Cloudflare Vectorize。

同时展示：

来源文件

Chunk

similarity score

十二、反向测试

测试：

“这个文档完全没有提到的内容是什么？”

观察：

系统是否能够识别：

“知识库没有足够相关资料。”

不要让模型在没有 Context 的情况下胡编乱造。

十三、最终验收

本阶段完成后，整个 RAG 项目必须形成完整闭环：

上传文档

↓

R2

↓

Parser

↓

Text Cleaning

↓

Chunk

↓

Gemini Embedding

↓

Cloudflare Vectorize

↓

用户 Query

↓

Gemini Query Embedding

↓

Vectorize Search

↓

Top-K

↓

Context

↓

RAG Prompt

↓

Gemini LLM

↓

Answer

↓

Sources

十四、最终 UI

最终页面：

左侧：

知识库文件管理、上传、删除、处理状态以及必要 RAG 参数。

右侧：

AI 对话区域。

AI 回答区域位于上方。

输入框固定在右侧底部。

支持 Markdown。

AI 回答下方可以展开查看参考资料。

整体 UI 保持简洁、现代、专业，不要设计成复杂后台系统。

十五、本阶段完成后的要求

完成后不要继续增加功能。

不要实现：

- Rerank
- Query Rewrite
- Hybrid Search
- Web Search
- Agent
- 多 Agent
- OCR
- 多租户
- 权限系统
- Redis
- Kafka
- 微服务
- AI Search
- 复杂工作流

本项目到此结束。

最后请输出：

1. 完整项目结构。
2. 核心 RAG 数据流。
3. 每个模块的职责。
4. 所有 API。
5. Cloudflare 资源配置。
6. Gemini 配置。
7. 本地开发方法。

8. 部署方法。
9. 测试方法。
10. 一份适合面试时介绍这个项目的技术说明。

尤其需要说明：

“这个项目的 RAG 是如何自己实现的，而不是依赖 AI Search 或一站式 RAG 服务。”